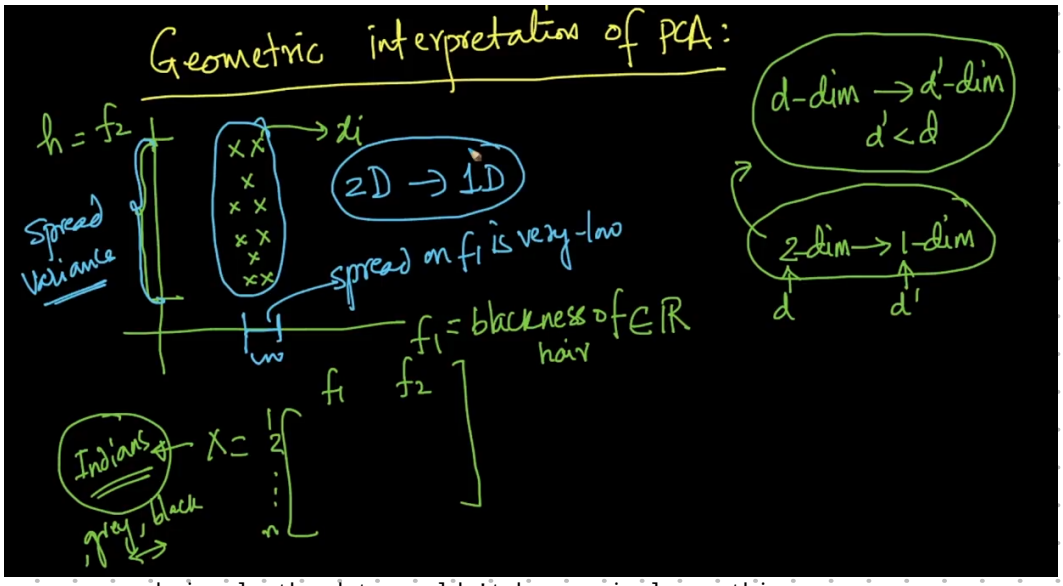
Principal Component Analysis



say we have data with 2 features,
f_1 = blackness of hair,
f_2 = height

we'd like to reduce the
2-D data to 1-D

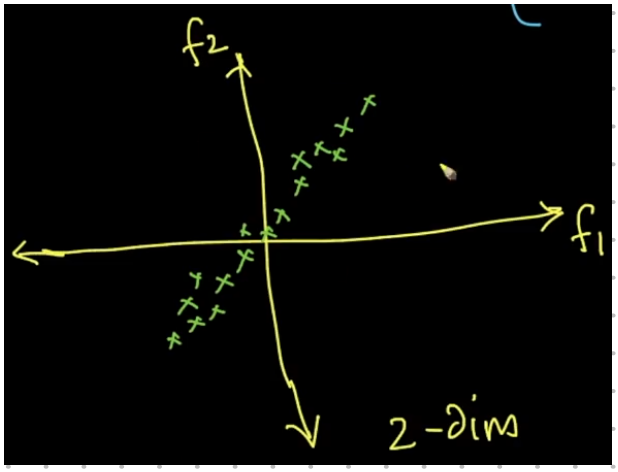
spread on the Y-axis is much more
than that on the X-axis.
i.e. variability is less in X

so if we are forced to skip a feature,
we'd skip f1

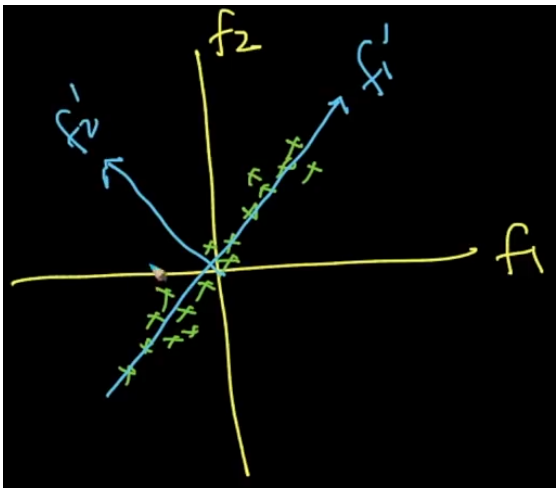
we'd assume (looking at the data, that
most of the people have the same black-
ness of hair, so we'll ignore it)

now obviously the data wouldn't be as simple as this

consider this data



this data is spread both on f1 and f2



consider this
2 new axes:
f1' and f2'

there's a lot of
spread on f1',
but spread on f2'
is much greater

so if we could project
xi's on f1'
then we could drop a feature

we want to find a direction f1' s.t.
the variance of xi's projected onto f1' is maximized

we only care about the direction,
so we'll find a u1, unit vector in that direction

we would project a point xi on u1 at xi'

$$x_i' = x_i \frac{u_1}{\|u_1\|} = u_1^T x_i \longrightarrow \vec{x_i'} = u_1^T \vec{x_i}$$

$$\mathcal{D} = \{x_i\}_{i=1}^n \quad \downarrow \quad \mathcal{D}' = \{x_i'\}_{i=1}^n$$

find u_1 s.t. $\text{Var}\{\text{proj}_{u_1} x_i\}_{i=1}^n$ is maximal.

$$\text{Var}\{u_1^T x_i\}_{i=1}^n = \frac{1}{n}\sum_{i=1}^n (u_1^T x_i - u_1^T \bar{x})^2$$

avg; x_i'; $\text{mean}\{x_i\}_{i=1}^n$

we find ourselves with an optimization problem

$$\text{Var}\{x_i'\}_{i=1}^n = \frac{1}{n}\sum_{i=1}^n (u_1^T x_i)^2$$

objective of an optimization problem

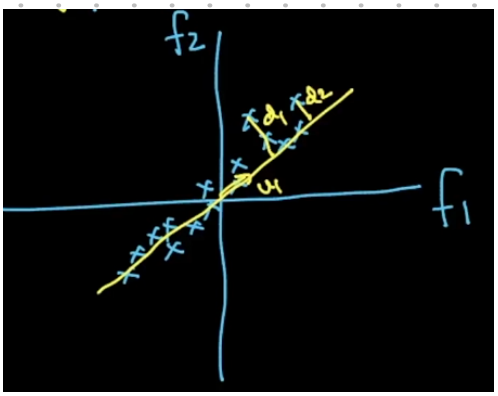
$$\max_{u_1} \frac{1}{n}\sum_{i=1}^n (u_1^T x_i)^2 \quad \text{Var}\{x_i'\}$$

Data-matrix

$$\text{s.t. } u_1^T u = 1 = \|u\|^2$$

Constraint

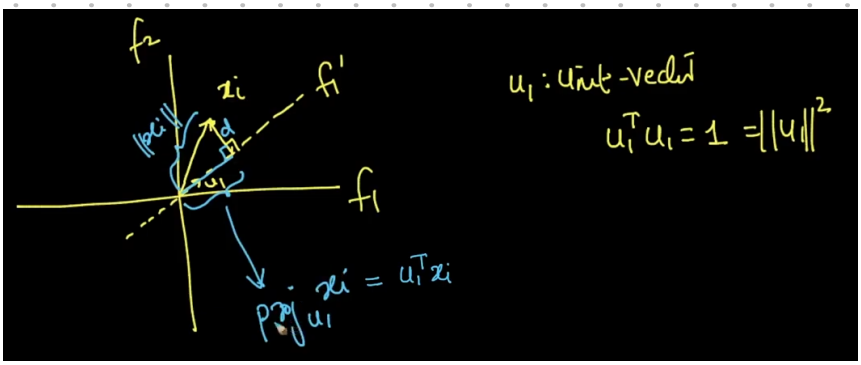
Alternative formulation: Distance minimization



for each data point xi, we can find the distance di from xi to u1

we want to find u1 which minimizes the squared distance

$$\min_{u_1} \sum_{i=1}^{2} d_i^2$$



$$d_i^2 = \|x_i\|^2 - (u_1^T x_i)^2$$
$$= x_i^T x_i - (u_1^T x_i)^2$$

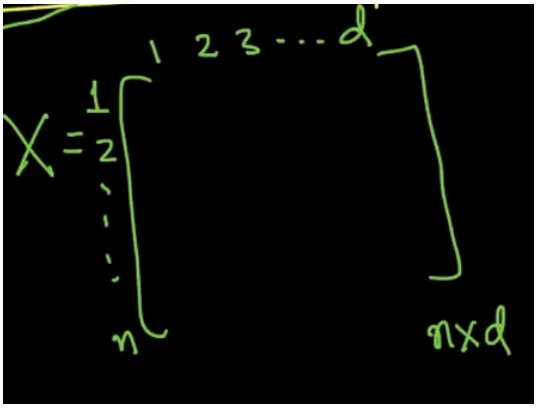
so we want min sum of di^2 i.e.

$$\min_{u_1} \sum_{i=1}^{n} \left(x_i^T x_i - (u_1^T x_i)^2 \right)$$

$$s.t \quad u^T u = 1$$

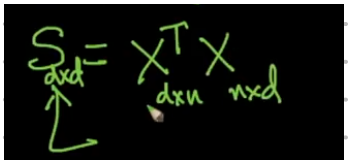
again we have an optimization problem

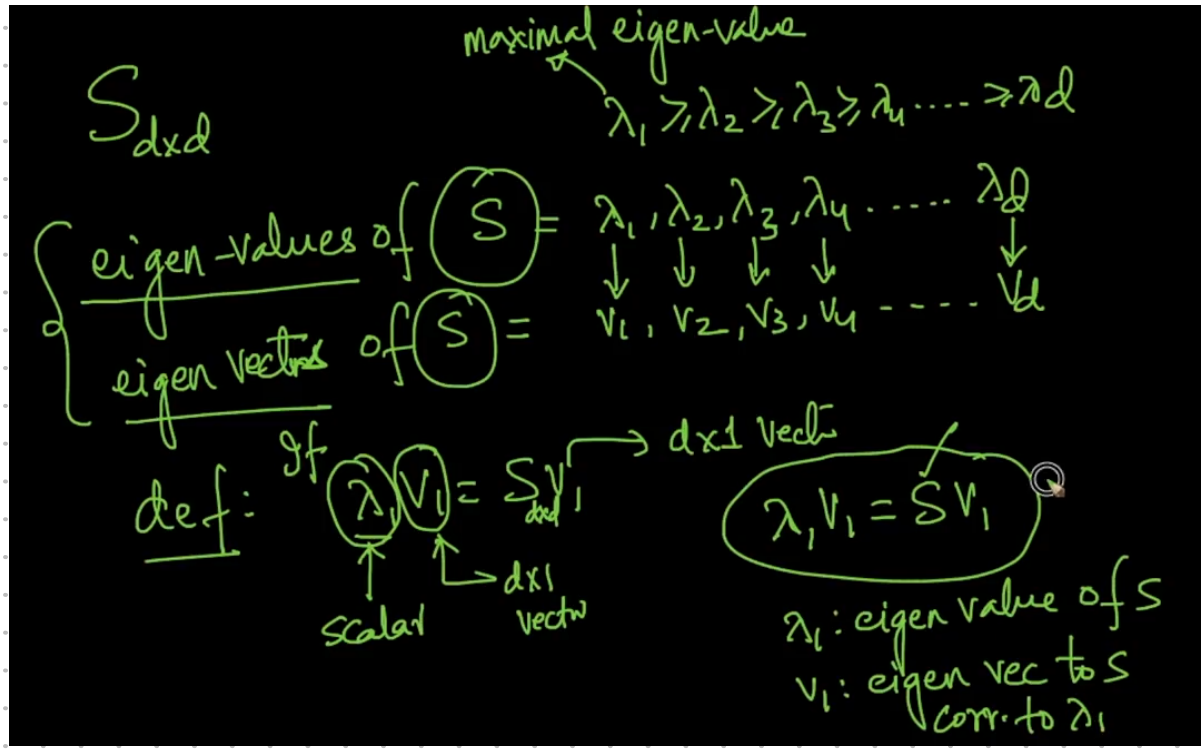
Solution to optimization problems:



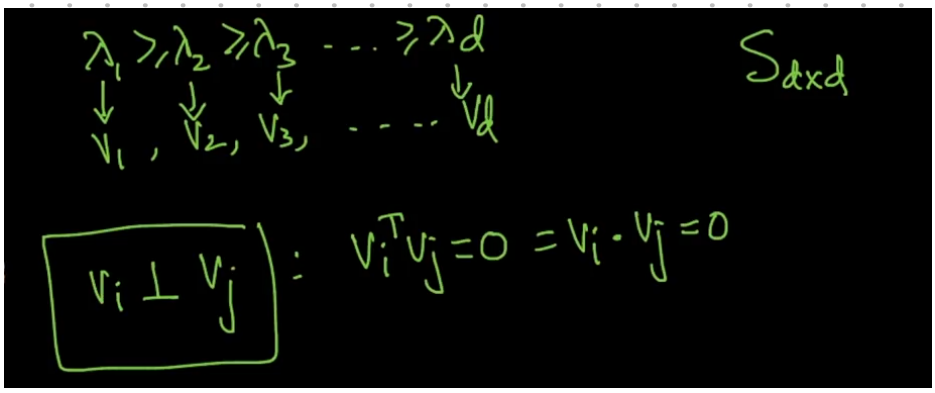
we have our data X, say X is standardized
for each column, mean = 0, std dev = 1

so we can get covariance matrix S as

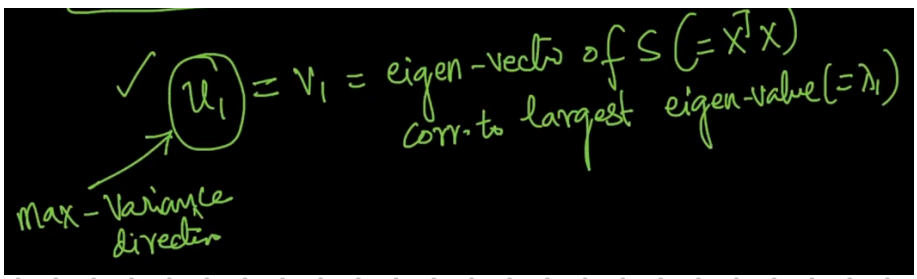




property: any 2 eigen-vectors are perpendicular to one-another

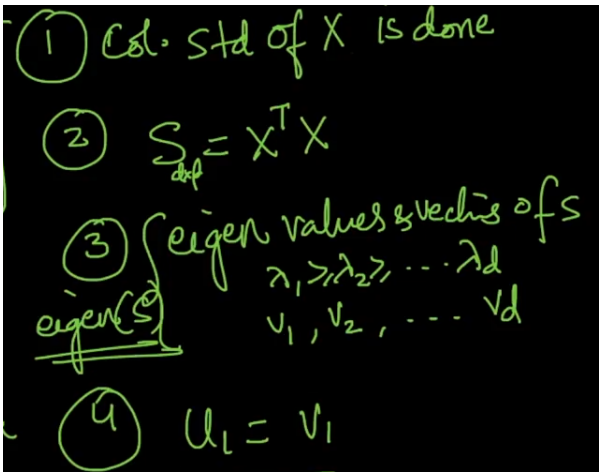


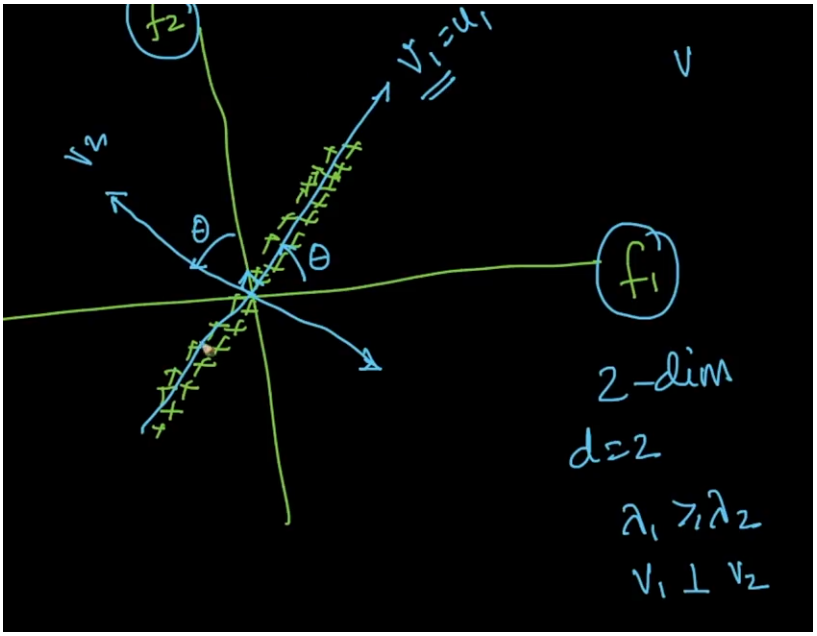
now for our optimization problem,



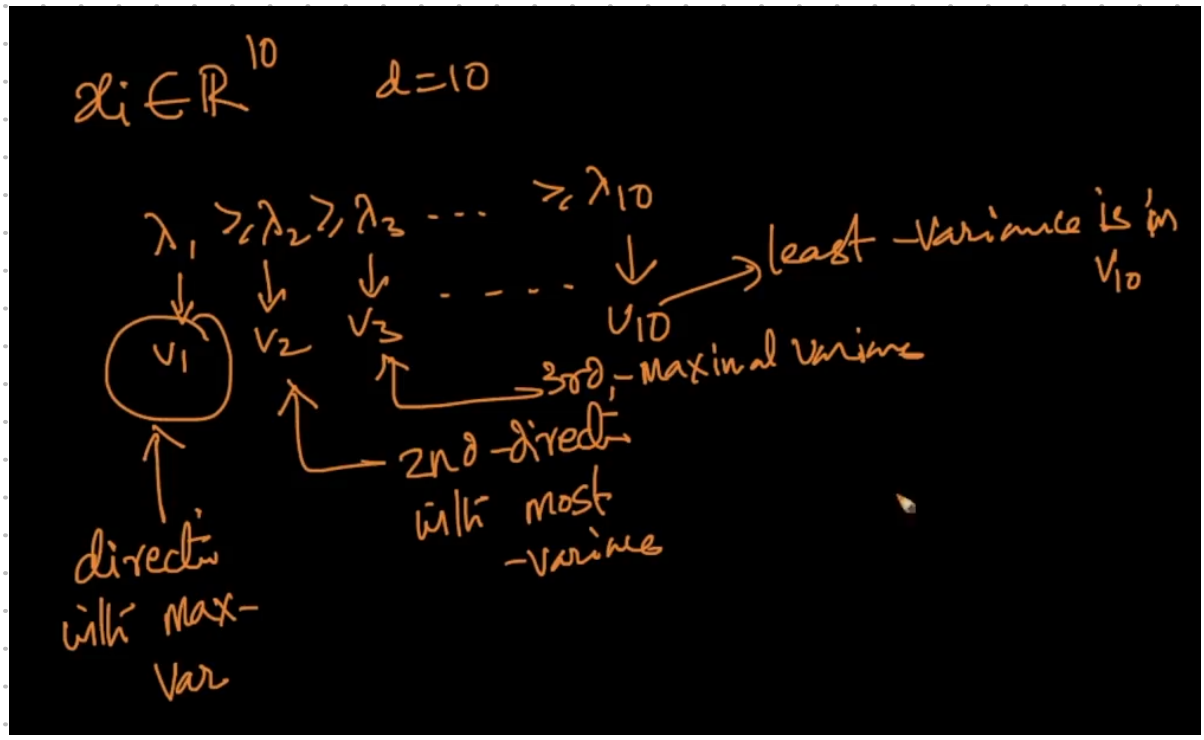
steps:
given X

column standardize X
find eigenpairs

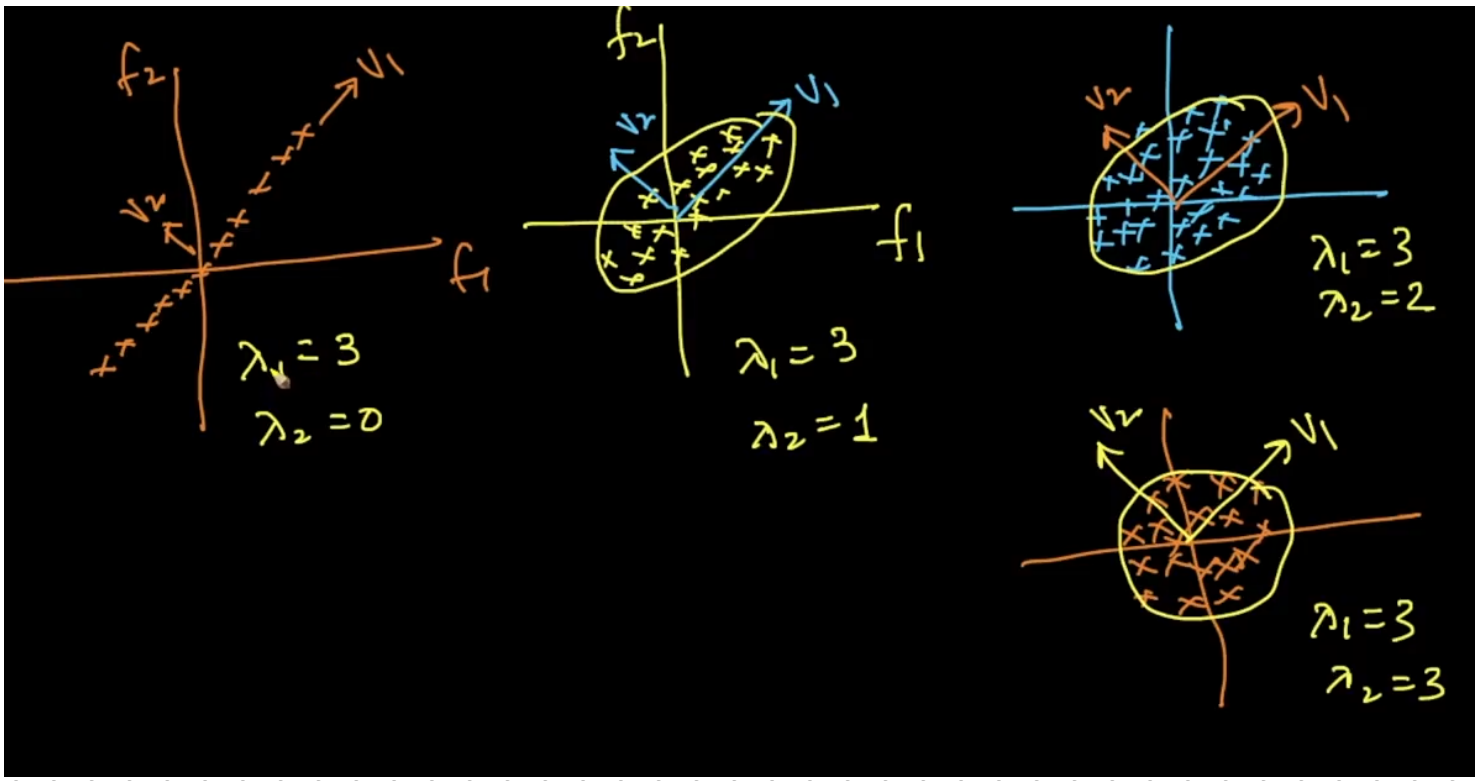




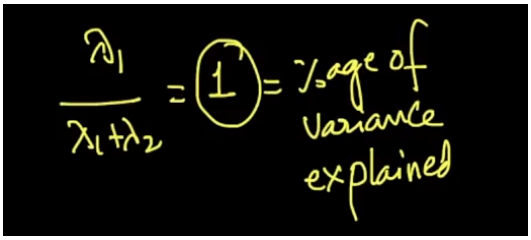
if we have 10-dim data



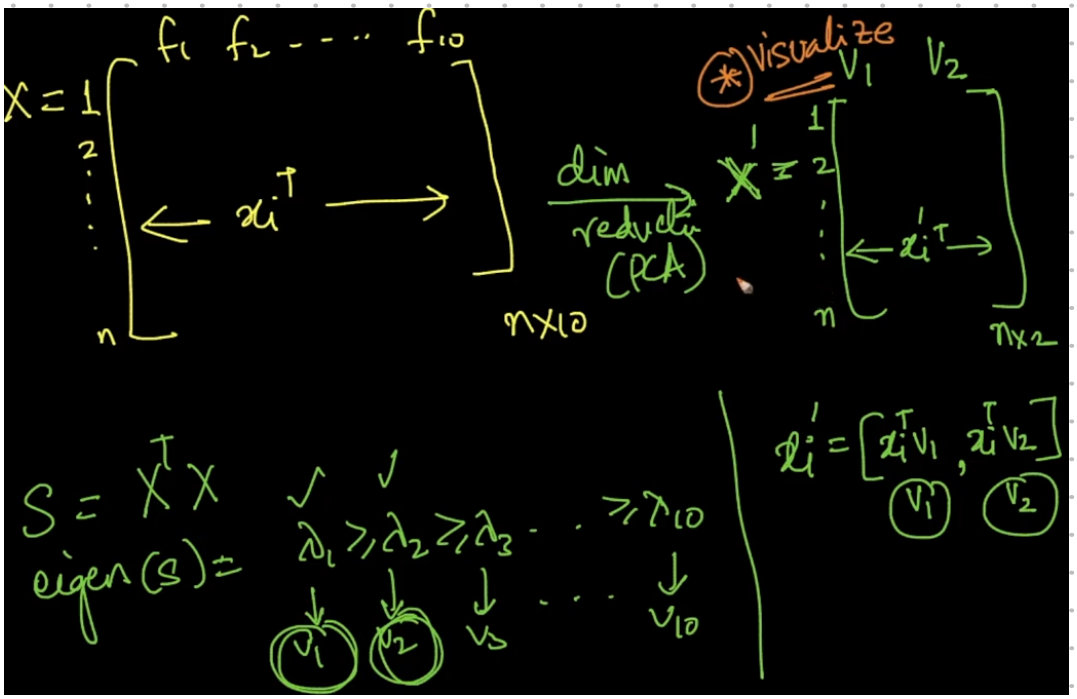
the eigen vectors give directions with maximal variance in desc order
what do we interpret by eigen values



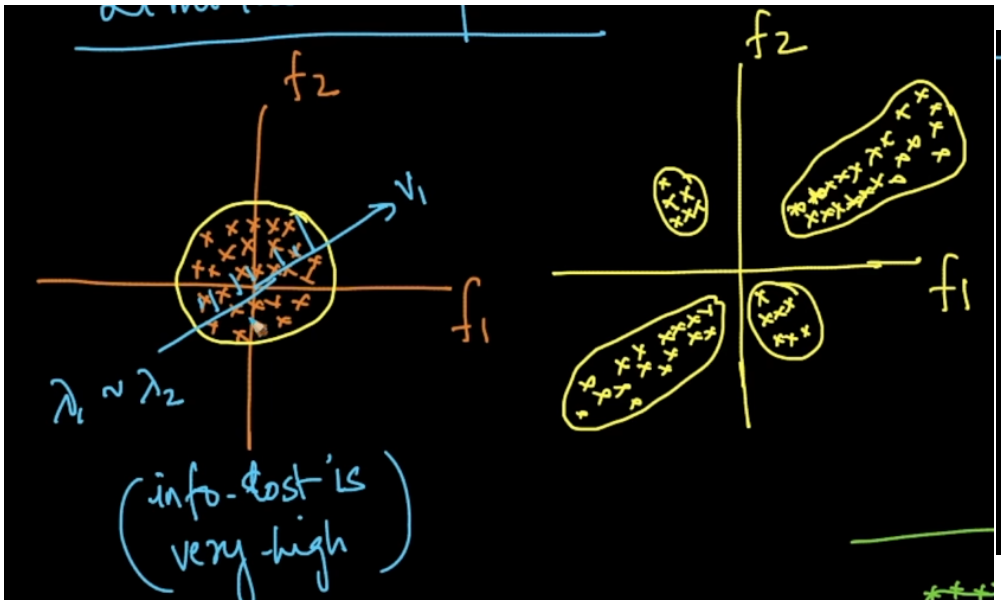
the data might not be perfectly aligned with v1
it might have some spread

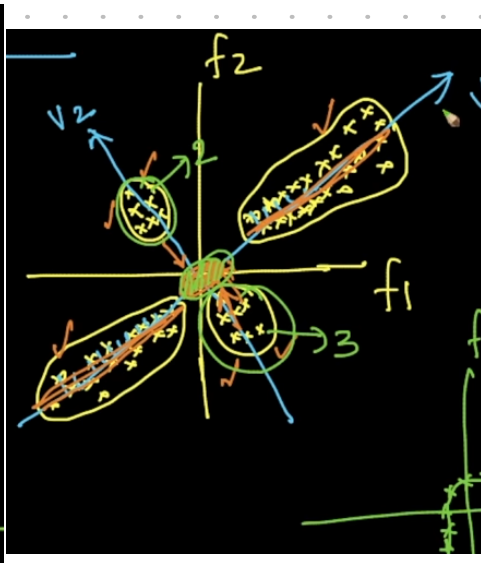


$\frac{\lambda_i}{\sum\lambda}$ = percentage of variance explained by ith eigen vector



Limitations of PCA:

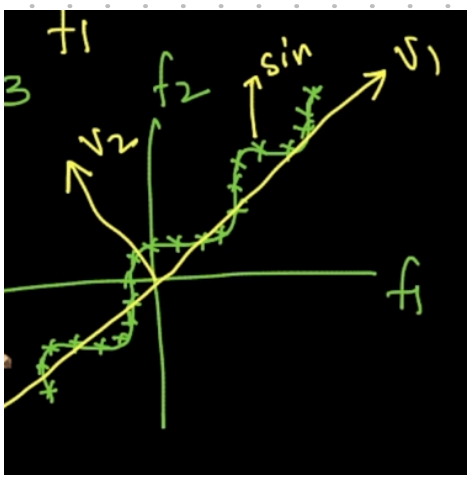




in this case,
where data is circular
or hyper-spherical

loss of data is high

in clusters like these, some clusters might be lost
clusters are not distinguished



characteristics like that of this sine wave would get lost in PCA

PCA

we have the MNIST dataset as our data

```
# we have features in d, corresp. labels in l
print(d.shape, l.shape)
# 784 features for 42k data points
```

[12] ✓ 0.0s

```
(42000, 784) (42000,)
```

```
labels = l
data = d
```

[13] ✓ 0.0s

we standardize our data

```
# data preprocessing
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
standardized_data = scaler.fit_transform(data)
print(standardized_data.shape)
```

✓ 0.5s

```
(42000, 784)
```

find covariance matrix

```
# covariance matrix
s = standardized_data
cov_matrix = np.matmul(s.T, s)
print(cov_matrix.shape)
```

✓ 2.3s

```
(784, 784)
/tmp/ipykernel_9477/3365637964.py:3: RuntimeWarning: invalid value encountered in matmul
  cov_matrix = np.matmul(s.T, s)
```

compute eigen pairs

```
# we need \lambda_1 and \lambda_2
from scipy.linalg import eigh
eig_values, eig_vecs = eigh(cov_matrix, subset_by_index=(782, 783)) # indexed as 0, 1, 2, ..., 783 in asc order, we need 2 largest

eig_vecs = eig_vecs.T
print(eig_vecs.shape)
```

✓ 1.4s

```
(2, 784)
```

new data = eigen vector * old data

```
new_data = np.matmul(eig_vecs, data.T)  # (2, 784) x (784, 42000)
print(new_data.shape)
```

✓ 0.2s

```
(2, 42000)
```

```
new_data = np.vstack((new_data, labels)).T
```
✓ 0.0s

```
print(new_data.shape)
```
✓ 0.0s

```
(42000, 3)
```

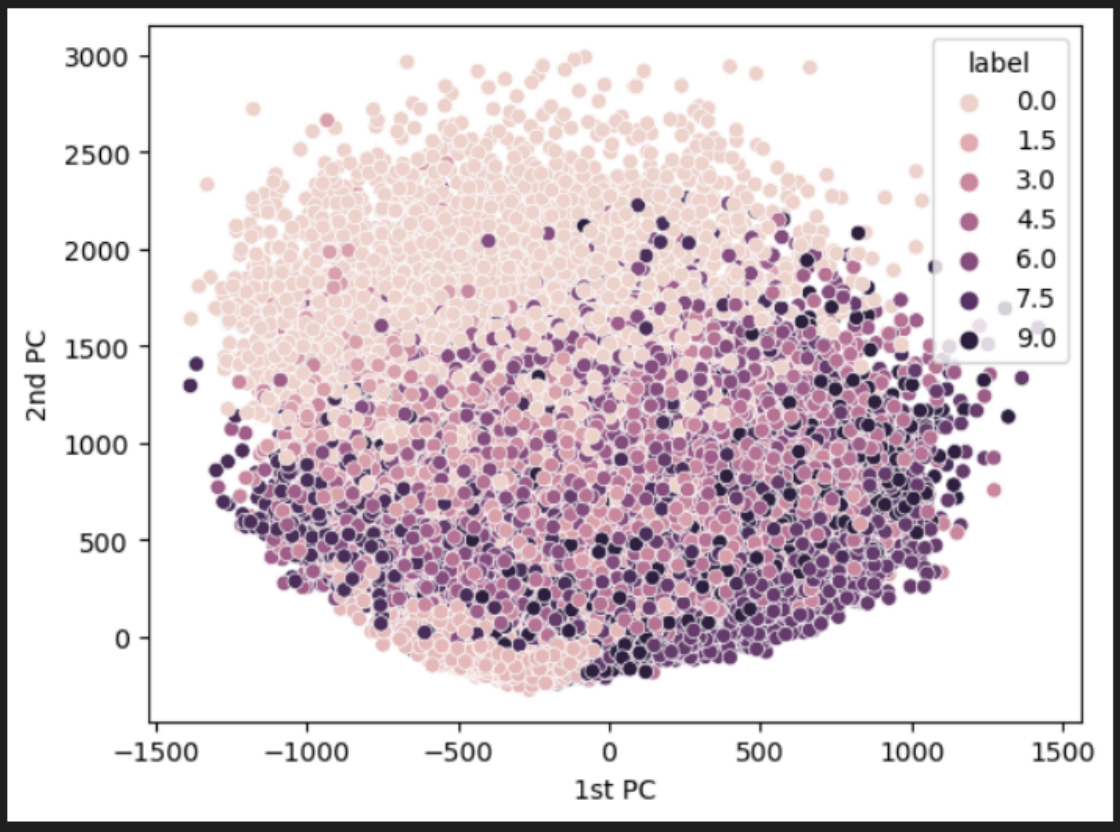
```
df = pd.DataFrame(data=new_data, columns=["1st PC", "2nd PC", "label"])
print(df.head)
```
✓ 0.0s

```
<bound method NDFrame.head of            1st PC       2nd PC  label
0     -540.331669   122.904004    1.0
1      339.624235  2318.125463    0.0
2     -261.521516  -112.964529    1.0
3      304.388819   539.841540    4.0
4      300.353405  2734.887930    0.0
...           ...          ...    ...
41995 -157.035887  1577.937985    0.0
41996 -209.502218  -241.544074    1.0
41997  422.015361   713.056694    7.0
41998 -478.970451   851.474058    6.0
41999   36.036059   199.028323    9.0

[42000 rows x 3 columns]>
```

```
sns.scatterplot(df, x="1st PC", y="2nd PC", hue="label")
```
✓ 12.1s

```
<AxesSubplot: xlabel='1st PC', ylabel='2nd PC'>
```



All of that can be done directly using the PCA function in sklearn

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
```
✓ 0.0s

```
data = d
labels = l
```
✓ 0.0s

```
PCA_data = pca.fit_transform(data)
print(PCA_data.shape)
```
✓ 3.4s

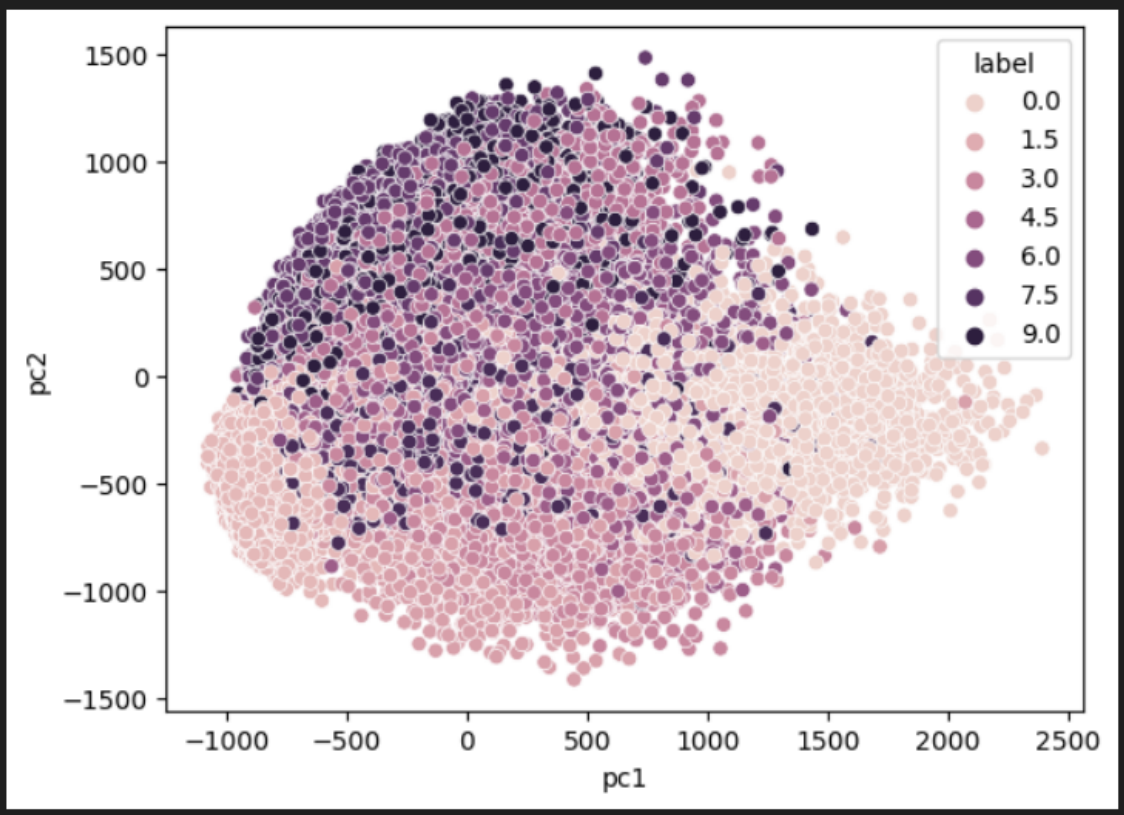
```
(42000, 2)
```

```
pca_data = np.vstack((PCA_data.T, labels)).T
print(pca_data.shape)
```
✓ 0.0s

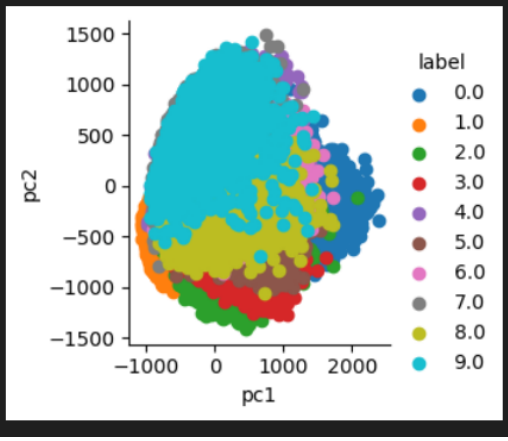
```
(42000, 3)
```

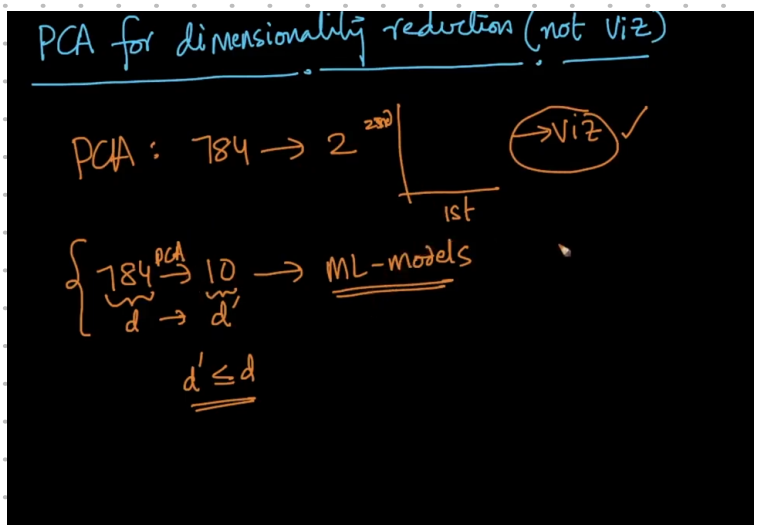
```
pca_df = pd.DataFrame(pca_data, columns=["pc1", "pc2", "label"])
sns.scatterplot(pca_df, x="pc1", y="pc2", hue="label")
```
✓ 12.7s

```
<AxesSubplot: xlabel='pc1', ylabel='pc2'>
```




```
sns.FacetGrid(pca_df, hue="label").map(plt.scatter, "pc1", "pc2").add_legend()
plt.show()
```
✓ 2.6s





PCA for dimensionality redcution (not for visualization)

```
[ ]  # PCA for dimensionality redcution (non-visualization)

     pca.n_components = 784
     pca_data = pca.fit_transform(sample_data)

     percentage_var_explained = pca.explained_variance_ / np.sum(pca.explained_variance_);

     cum_var_explained = np.cumsum(percentage_var_explained)

     # Plot the PCA spectrum
     plt.figure(1, figsize=(6, 4))

     plt.clf()
     plt.plot(cum_var_explained, linewidth=2)
     plt.axis('tight')
     plt.grid()
     plt.xlabel('n_components')
     plt.ylabel('Cumulative_explained_variance')
     plt.show()


     # If we take 200-dimensions, approx. 90% of variance is expalined.
```

